Tarea Práctica — Clase 5

Implementación de proyecto dbt con DuckDB y MotherDuck

Nicolás Blanco · Introducción a la Ingeniería de Datos · MIA 03

Fuentes: pokemon + onecall → MotherDuck → dbt

En esta práctica se configuró un proyecto de **dbt** utilizando **dbt-duckdb** sobre **Ubuntu/WSL**, conectado a una base de datos en **MotherDuck**. A partir de las tablas fuente disponibles se construyó un flujo por capas (`sources`, `staging`, `intermediate` y `marts`), se ejecutaron los modelos y se generó la documentación con su grafo de dependencias.

Entorno de trabajo

La implementación se realizó en **Ubuntu dentro de WSL** en Windows. Esta decisión permitió mantener el trabajo dentro del mismo entorno donde ya se estaba ejecutando Airbyte, evitando problemas de rutas, duplicación de entornos Python y diferencias entre herramientas de Windows y Linux.

Instalación del entorno virtual

Primero se instalaron los paquetes necesarios para crear entornos virtuales y usar `pip`:

```
sudo apt update
sudo apt install -y python3-pip python3-venv
```

Luego se creó y activó el entorno virtual:

```
python3 -m venv ~/venvs/dbt-env
source ~/venvs/dbt-env/bin/activate
```

Instalación de dbt

Dentro del entorno virtual se instaló `dbt-duckdb`:

```
python -m pip install --upgrade pip
python -m pip install dbt-duckdb
dbt --version
```

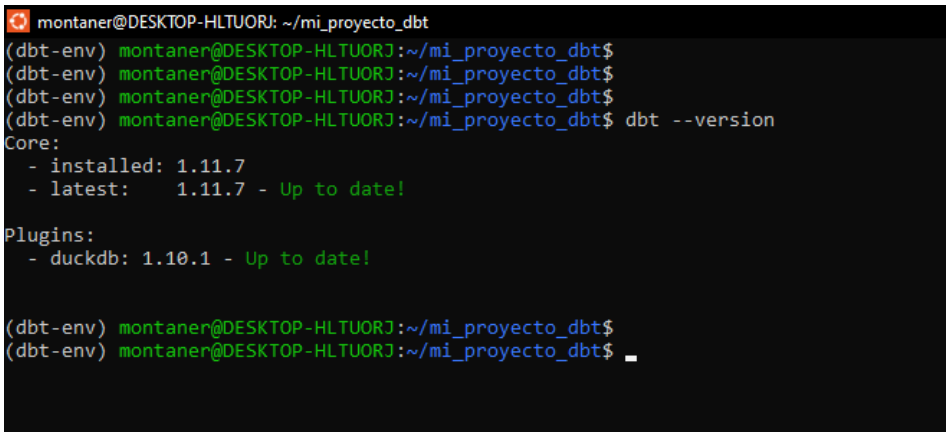


Figura 1: Verificación de instalación de dbt en Ubuntu/WSL

Creación del proyecto dbt

Una vez instalado dbt, se creó el proyecto con el comando:

```
dbt init mi_proyecto_dbt
```

Durante el asistente de inicialización se seleccionó el adaptador `duckdb`. El proyecto generado incluyó la estructura base típica de dbt, con carpetas como `models`, `macros`, `tests`, `snapshots` y `seeds`.

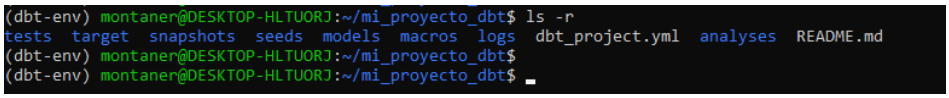


Figura 2: Estructura base del proyecto generado por dbt

Configuración de conexión a MotherDuck

El archivo `profiles.yml`, ubicado en `~/.dbt/profiles.yml`, fue ajustado para conectar el proyecto con una base de datos en MotherDuck.

La configuración utilizada fue la siguiente:

```
mi_proyecto_dbt:
  outputs:
    dev:
      type: duckdb
      path: "md:my_db"
      extensions:
        - motherduck
      motherduck_token: "{{ env_var('MOTHERDUCK_TOKEN') }}"
      threads: 1

  target: dev
```

Además, se exportó el token de acceso a MotherDuck como variable de entorno:

```
export MOTHERDUCK_TOKEN="TU_TOKEN_REAL"
```

Finalmente, la conexión fue validada con:

```
dbt debug
```

La prueba resultó exitosa y confirmó que tanto el proyecto como el perfil estaban correctamente configurados.

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cat ~/.dbt/profiles.yml
mi_proyecto_dbt:
  outputs:
    dev:
      type: duckdb
      path: "md:my_db"
      extensions:
        - motherduck
      motherduck_token: "{{ env_var('MOTHERDUCK_TOKEN') }}"
      threads: 1

  target: dev
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 3: Configuración de conexión a MotherDuck en `profiles.yml`

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ dbt debug
15:37:24  Running with dbt=1.11.7
15:37:24  dbt version: 1.11.7
15:37:24  python version: 3.12.3
15:37:24  python path: /home/montaner/venvs/dbt-env/bin/python
15:37:24  os info: Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-with-glibc2.39
15:37:25  Using profiles dir at /home/montaner/.dbt
15:37:25  Using profiles.yml file at /home/montaner/.dbt/profiles.yml
15:37:25  Using dbt_project.yml file at /home/montaner/mi_proyecto_dbt/dbt_project.yml
15:37:25  adapter type: duckdb
15:37:25  adapter version: 1.10.1
15:37:25  Configuration:
15:37:25    profiles.yml file [OK found and valid]
15:37:25    dbt_project.yml file [OK found and valid]
15:37:25  Required dependencies:
15:37:25   - git [OK found]

15:37:25  Connection:
15:37:25    database: my_db
15:37:25    schema: main
15:37:25    path: md:my_db
15:37:25    config_options: None
15:37:25    extensions: ['motherduck']
15:37:25    settings: {}
15:37:25    external_root: .
15:37:25    use_credential_provider: None
15:37:25    attach: None
15:37:25    filesystems: None
15:37:25    remote: None
15:37:25    plugins: [{'module': 'motherduck', 'alias': None, 'config': None}]
15:37:25    disable_transactions: False
15:37:25  Registered adapter: duckdb=1.10.1
15:37:33    Connection test: [OK connection ok]

15:37:33  All checks passed!
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 4: Validación de conexión con MotherDuck

Definición de fuentes

Se creó el archivo `models/staging/_sources.yml` para declarar las tablas fuente del proyecto.

```yaml
version: 2

sources:
  - name: raw
    database: my_db
    schema: main
    tables:
      - name: pokemon
      - name: onecall
```

Este archivo permitió usar la función `source()` dentro de los modelos de dbt y establecer una referencia explícita a las tablas crudas disponibles en MotherDuck.

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cat models/staging/_sources.yml
version: 2

sources:
  - name: raw
    database: my_db
    schema: main
    tables:
      - name: pokemon
      - name: onecall
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 5: Definición de fuentes del proyecto

Capa staging

En la capa `staging` se construyeron modelos iniciales para normalizar y exponer los datos crudos de cada fuente.

Modelo `stg_pokemon`

```sql
WITH source AS (
    SELECT * FROM {{ source('raw', 'pokemon') }}
),

renamed AS (
    SELECT
        id AS pokemon_id,
        name AS pokemon_name,
        height,
        weight,
        base_experience,
        types,
        _airbyte_extracted_at AS loaded_at
    FROM source
)

SELECT * FROM renamed
```

Modelo `stg_weather`

```sql
WITH source AS (
    SELECT * FROM {{ source('raw', 'onecall') }}
)

SELECT * FROM source
```

Luego se ejecutaron los modelos de esta capa con:

```
dbt run --select staging.*
```

La primera ejecución mostró un error debido a que la tabla fuente había sido nombrada inicialmente como `weather`, cuando el nombre correcto en MotherDuck era `onecall`. Una vez corregido `_sources.yml`, la ejecución fue exitosa.

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cat models/staging/stg_pokemon.sql
WITH source AS (
    SELECT * FROM {{ source('raw', 'pokemon') }}
),

renamed AS (
    SELECT
        id AS pokemon_id,
        name AS pokemon_name,
        height,
        weight,
        base_experience,
        types,
        _airbyte_extracted_at AS loaded_at
    FROM source
)

SELECT * FROM renamed
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 6: Modelo de staging para la fuente pokemon

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
els/sta(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cat models/staging/stg_weather.sql
WITH source AS (
    SELECT * FROM {{ source('raw', 'onecall') }}
)

SELECT * FROM source
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 7: Modelo de staging para la fuente onecall

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
 --selec(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ dbt run --select staging.*
15:39:36  Running with dbt=1.11.7
15:39:37  Registered adapter: duckdb=1.10.1
15:39:38  Found 8 models, 4 data tests, 2 sources, 474 macros
15:39:38
15:39:38  Concurrency: 1 threads (target='dev')
15:39:38
15:39:42  1 of 2 START sql view model main.stg_pokemon ................................ [RUN]
15:39:43  1 of 2 OK created sql view model main.stg_pokemon ........................... [OK in 1.40s]
15:39:43  2 of 2 START sql view model main.stg_weather ................................ [RUN]
15:39:44  2 of 2 OK created sql view model main.stg_weather ........................... [OK in 1.46s]
15:39:44
15:39:44  Finished running 2 view models in 0 hours 0 minutes and 6.93 seconds (6.93s).
15:39:44
15:39:44  Completed successfully
15:39:44
15:39:44  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 8: Ejecución de la capa staging

Capa intermediate

La capa `intermediate` se utilizó para aplicar una transformación adicional sobre los datos provenientes de `staging`.

Modelo `int_pokemon_with_types`

```
WITH pokemon AS (
    SELECT * FROM {{ ref('stg_pokemon') }}
),

pokemon_types AS (
    SELECT
        pokemon_id,
        pokemon_name,
        height,
```

```sql
        weight,
        base_experience,
        types->0->'type'->>'name' AS type_primary,
        types->1->'type'->>'name' AS type_secondary
    FROM pokemon
)

SELECT * FROM pokemon_types
```

Modelo int_weather

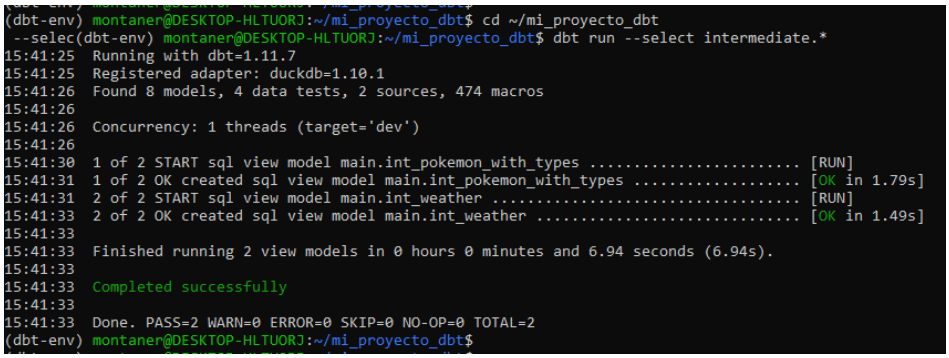
```sql
WITH weather AS (
    SELECT * FROM {{ ref('stg_weather') }}
),

renamed AS (
    SELECT
        lat,
        lon,
        current,
        hourly,
        daily,
        alerts
    FROM weather
)

SELECT * FROM renamed
```

Ambos modelos fueron ejecutados con:

```sql
dbt run --select intermediate.*
```

Figura 9: Modelo intermedio para pokemon

Figura 10: Modelo intermedio para onecall



```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
 --selec(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ dbt run --select intermediate.*
15:41:25  Running with dbt=1.11.7
15:41:25  Registered adapter: duckdb=1.10.1
15:41:26  Found 8 models, 4 data tests, 2 sources, 474 macros
15:41:26
15:41:26  Concurrency: 1 threads (target='dev')
15:41:26
15:41:30  1 of 2 START sql view model main.int_pokemon_with_types ........................ [RUN]
15:41:31  1 of 2 OK created sql view model main.int_pokemon_with_types ................... [OK in 1.79s]
15:41:31  2 of 2 START sql view model main.int_weather ................................... [RUN]
15:41:33  2 of 2 OK created sql view model main.int_weather ............................... [OK in 1.49s]
15:41:33
15:41:33  Finished running 2 view models in 0 hours 0 minutes and 6.94 seconds (6.94s).
15:41:33
15:41:33  Completed successfully
15:41:33
15:41:33  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 11: Ejecución de la capa intermediate

Capa marts

Finalmente, se construyeron los modelos de salida de la capa `marts`.

Modelo `obt_pokemon`

```sql
WITH pokemon AS (
    SELECT * FROM {{ ref('int_pokemon_with_types') }}
),

final AS (
    SELECT
        pokemon_id,
        pokemon_name,
        height,
        weight,
        base_experience,
        type_primary,
        type_secondary,
        CASE
            WHEN base_experience >= 200 THEN 'Legendary'
            WHEN base_experience >= 100 THEN 'Strong'
            ELSE 'Normal'
        END AS power_tier
    FROM pokemon
)

SELECT * FROM final
```

Modelo `obt_weather`

```sql
WITH weather AS (
    SELECT * FROM {{ ref('int_weather') }}
),

final AS (
    SELECT
        lat,
        lon,
        current,
        hourly,
        daily,
        alerts
    FROM weather
)
```

```sql
SELECT * FROM final
```

La capa final fue ejecutada mediante:

```
dbt run --select marts.*
```

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cat models/marts/obt_pokemon.sql
WITH pokemon AS (
    SELECT * FROM {{ ref('int_pokemon_with_types') }}
),

final AS (
    SELECT
        pokemon_id,
        pokemon_name,
        height,
        weight,
        base_experience,
        type_primary,
        type_secondary,
        CASE
            WHEN base_experience >= 200 THEN 'Legendary'
            WHEN base_experience >= 100 THEN 'Strong'
            ELSE 'Normal'
        END AS power_tier
    FROM pokemon
)

SELECT * FROM final
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 12: Modelo mart para pokemon

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cat models/marts/obt_weather.sql
WITH weather AS (
    SELECT * FROM {{ ref('int_weather') }}
),

final AS (
    SELECT
        lat,
        lon,
        current,
        hourly,
        daily,
        alerts
    FROM weather
)

SELECT * FROM final
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 13: Modelo mart para onecall

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ cd ~/mi_proyecto_dbt
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$ dbt run --select marts.*
15:46:59  Running with dbt=1.11.7
15:46:59  Registered adapter: duckdb=1.10.1
15:47:00  Found 8 models, 4 data tests, 2 sources, 474 macros
15:47:00
15:47:00  Concurrency: 1 threads (target='dev')
15:47:00
15:47:03  1 of 2 START sql view model main.obt_pokemon ................................ [RUN]
15:47:05  1 of 2 OK created sql view model main.obt_pokemon ........................... [OK in 1.41s]
15:47:05  2 of 2 START sql view model main.obt_weather ................................ [RUN]
15:47:06  2 of 2 OK created sql view model main.obt_weather ........................... [OK in 1.68s]
15:47:06
15:47:06  Finished running 2 view models in 0 hours 0 minutes and 6.51 seconds (6.51s).
15:47:06
15:47:06  Completed successfully
15:47:07
15:47:07  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt$
```

Figura 14: Ejecución de la capa marts

Documentación del proyecto y DAG

Una vez construidos todos los modelos, se generó la documentación del proyecto con:

```
dbt docs generate
dbt docs serve --host 0.0.0.0 --port 8080
```

Dado que el proyecto se ejecutó en WSL, el acceso desde el navegador no funcionó correctamente con `localhost`. En consecuencia, la documentación fue visualizada utilizando la IP interna de Ubuntu:

```
http://192.168.38.219:8080
```

En la documentación generada se pudo visualizar correctamente el DAG del proyecto, incluyendo las dependencias entre `sources`, `staging`, `intermediate` y `marts`.

El grafo final mostró dos pipelines principales:

- `raw.pokemon` → *stg_pokemon* → *int_pokemon_with_types* → *obt_pokemon*
- `raw.onecall` → *stg_weather* → *int_weather* → *obt_weather*

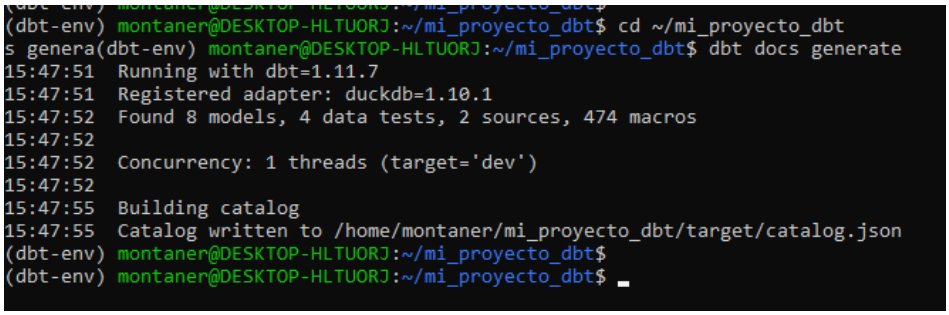


Figura 15: Generación de documentación dbt

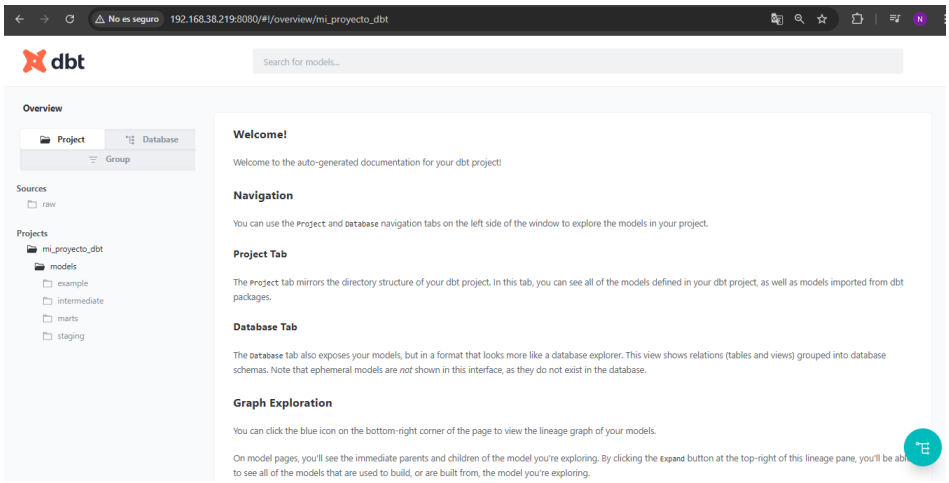


Figura 16: Vista general de la documentación dbt

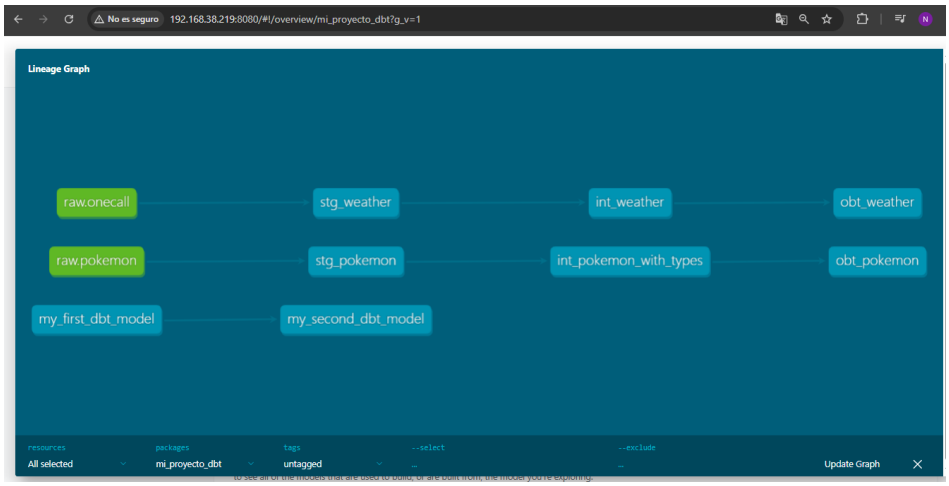


Figura 17: Lineage graph final del proyecto

Conclusión

La práctica permitió implementar correctamente un flujo básico de trabajo con dbt, integrando una base en MotherDuck y organizando el proyecto bajo una arquitectura por capas. Se logró:

- Instalar dbt en un entorno Ubuntu/WSL.
- Conectar el proyecto a MotherDuck.
- Definir tablas fuente.
- Construir modelos de `staging`. `intermediate` y `marts`.
- Ejecutar satisfactoriamente todas las capas.
- Generar la documentación y visualizar el DAG resultante.

Como resultado final, se obtuvo un proyecto funcional y documentado, con dos flujos paralelos de transformación de datos, uno para `pokemon` y otro para `onecall`. La práctica también permitió reforzar conceptos fundamentales de dbt, como el uso de `source()`, `ref()`, separación por capas y visualización de dependencias entre modelos.